TECNOLOGÍA DE COMPUTADORES

Tema 1: Diseño Hardware con VHDL

PROF. ANGEL BRIZ



UNIVERSIDAD NEBRIJA

ANTERIORMENTE…

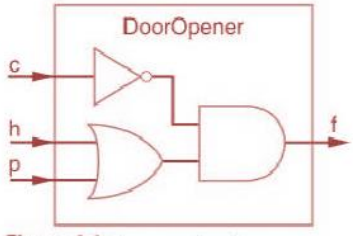
En la asignatura “Sistemas Digitales” se estudiaron:

- Fundamentos del Álgebra de Boole y mapas de Karnaugh
- Puertas lógicas y circuitos combinacionales
- Circuitos secuenciales y máquinas de estado

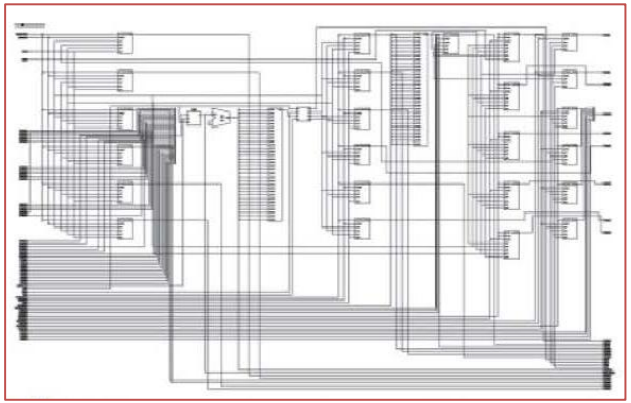
En Tecnología de Computadores aprenderemos diseñar circuitos digitales utilizando lenguajes de descripción hardware (HDL).

DESCRIPCION DE CIRCUITOS DIGITALES

- La descripcion de circuitos usando esquematicos es util *solo* cuando se tienes unos pocos componentes.



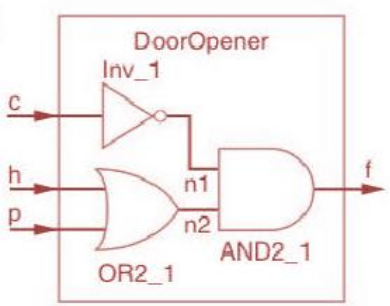
- Los esquematicos continen mucha información inutil. Tamaño de los componentes, forma, posición, lineas, etc.



- Cuando el circuito es mas complejo, el tiempo de generación aumenta significativamente. Ademas, la información que aporta al usuario decrece.

DESCRIPCION DE CIRCUITOS DIGITALES

- Una descripcion usando texto, reduciria la cantidad de informacion “inutil” requerida para dibujar un esquematico.



We'll now describe a circuit whose name is DoorOpener.
The external inputs are c, h, and p, which are bits.
The external output is f, which is a bit.

We assume you know the behavior of these components:
An inverter, which has a bit input x, and bit output F.
A 2-input OR gate, which has bit inputs x and y, and bit output F.
A 2-input AND gate, which has bit inputs x and y, and bit output F.

The circuit has internal wires n1 and n2, both bits.
The DoorOpener circuit internally consists of:
An inverter named Inv_1, whose input x connects to external input c, and whose output connects to n1.
A 2-input OR gate named OR2_1, whose inputs connect to external inputs h and p, and whose output connects to n2.
A 2-input AND gate named AND2_1, whose inputs connect to n1 and n2, and whose output connects to external output f.
That's all.

DESCRIPCION DE CIRCUITOS DIGITALES

- Para que una descripcion usando texto pueda ser leida por un computador, el “lenguaje” debe tener una sintaxis precisa y un significado preciso para cada construcción.

- Los **lenguajes de descripcion hardware** (*Hardware Description Language* – HDL) no solo permiten describir las conexiones entre componentes sino Tambien el comportamiento de los mismos.

LENGUAJES DE DESCRIPCION HARDWARE

Los mas ampliamente usados:

- VHDL
- Verilog / System Verilog

Sintesis de alto nivel (HLS) → mayor nivel de abstraccion:

- HDL coder (Matworks)
- Intel HLS compiler (Intel FPGA)
- Vivado HLS (AMD Xilinx)
- …

Producen codigo VHDL / Verilog

LENGUAJES DE DESCRIPCION HARDWARE

Los mas ampliamente usados:

- VHDL
- Verilog / System Verilog

Sintesis de alto nivel (HLS) → mayor nivel de abstraccion:

- HDL coder (Matworks)
- Intel HLS compiler (Intel FPGA)
- Vivado HLS (AMD Xilinx)
- …

Producen codigo VHDL / Verilog

INTRODUCCION A VHDL

- Lenguaje de **Descripción** Hardware definido por IEEE (*Institute of Electrical and Electronics Engineers*) en los años 80
  - VHDL: Contracción de las siglas VHSIC + HDL
  - VHSIC: Very High-Speed Integrated Circuit
  - HDL: Hardware **Description** Language

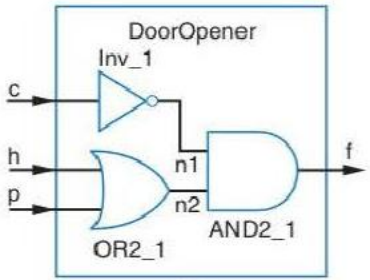
- VHDL nos permite definir un nivel de abstracción superior:
  - Describimos las entradas/salidas del circuito y su comportamiento
  - Un software genera las puertas lógicas y conexiones por nosotros
  - Podemos reutilizar circuitos ya descritos, facilitando la modularidad

INTRODUCCION A VHDL

- Reglas importantes sobre VHDL:
  - **NO ES UN LENGUAJE DE PROGRAMACION**
  - Describe hardware que se ejecuta en paralelo.
  - Se debe conocer previamente el circuito a diseñar (Papel → Ordenador)
  - El hardware descrito usando VHDL puede ser implementado en una FPGA o usado para fabricar un ASIC.

INTRODUCCION AL VHDL

- Volvamos al ejemplo inicial:



We'll now describe a circuit whose name is DoorOpener.
The external inputs are c, h, and p, which are bits.
The external output is f, which is a bit.

We assume you know the behavior of these components:
An inverter, which has a bit input x, and bit output F.
A 2-input OR gate, which has bit inputs x and y, and bit output F.
A 2-input AND gate, which has bit inputs x and y, and bit output F.

The circuit has internal wires n1 and n2, both bits.

The DoorOpener circuit internally consists of:
An inverter named Inv_1, whose input x connects to external input c, and whose output connects to n1.
A 2-input OR gate named OR2_1, whose inputs connect to external inputs h and p, and whose output connects to n2.
A 2-input AND gate named AND2_1, whose inputs connect to n1 and n2, and whose output connects to external output f.
That's all.

```
library ieee;
use ieee.std_logic_1164.all;
entity DoorOpener is
  port ( c, h, p: in std_logic;
         f: out std_logic
  );
end DoorOpener;

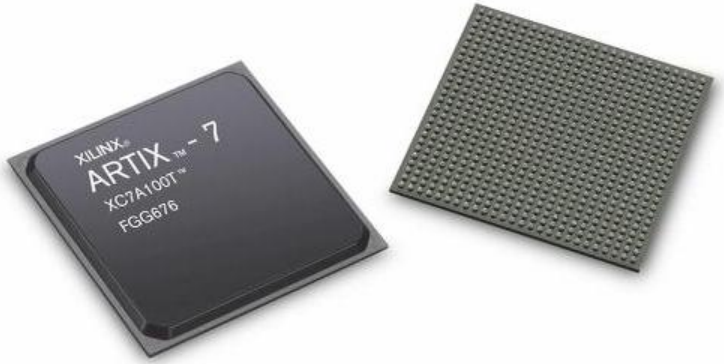
architecture Circuit of DoorOpener is
  component Inv
    port (x: in std_logic;
          F: out std_logic);
  end component;
  component OR2
    port (x,y: in std_logic;
          F: out std_logic);
  end component;
  component AND2
    port (x,y: in std_logic;
          F: out std_logic);
  end component;
  signal n1,n2: std_logic; --internal wires

begin
  Inv_1: Inv port map (x=>c, F=>n1);
  OR2_1: OR2 port map (x=>h,y=>p,F=>n2);
  AND2_1: AND2 port map (x=>n1,y=>n2,F=>f);
end Circuit;
```

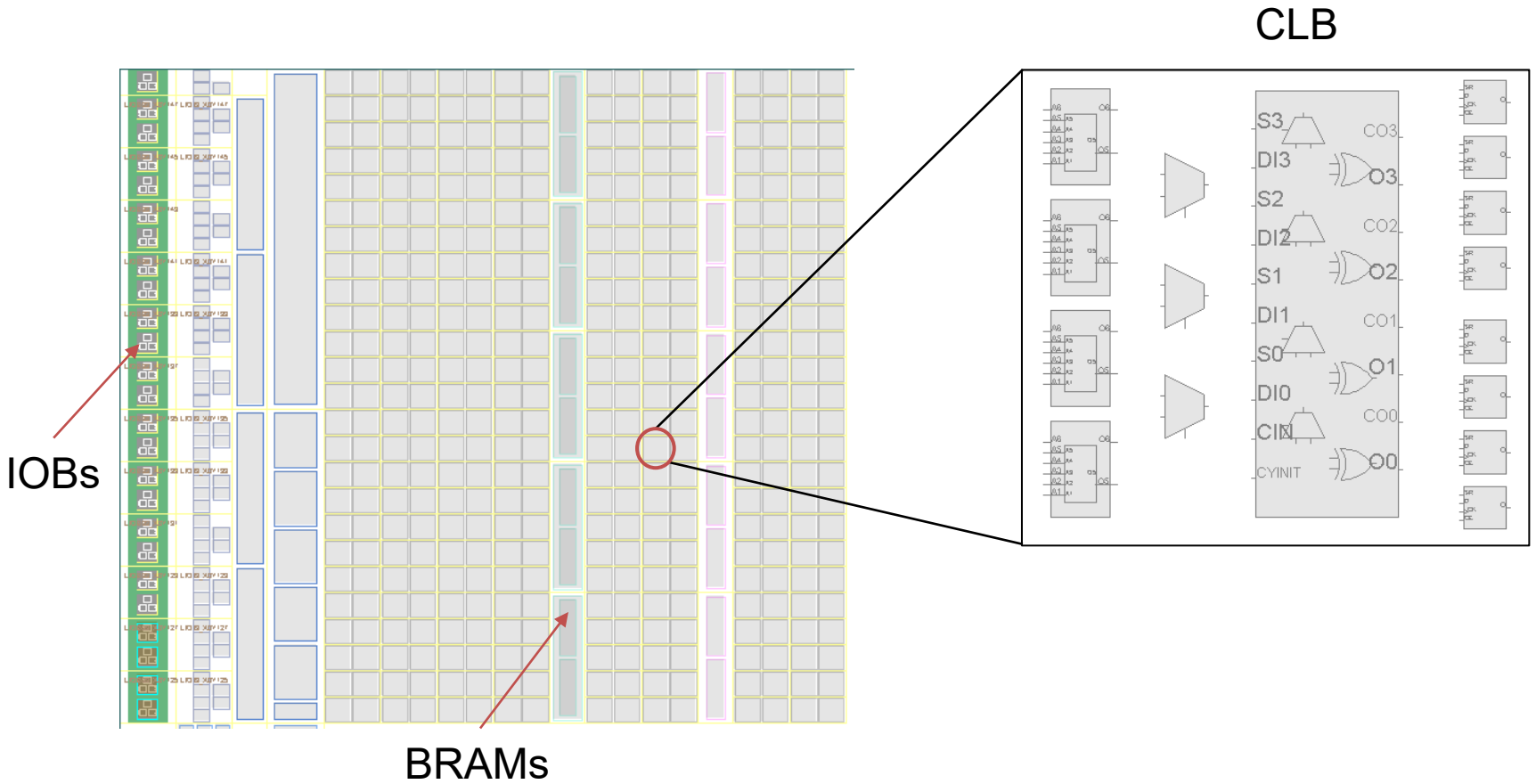
INTRODUCCION A LAS FPGAs

- Una FPGA (Field Programmable Gate Array) es un dispositivo semiconductor reprogramable tras su fabricación

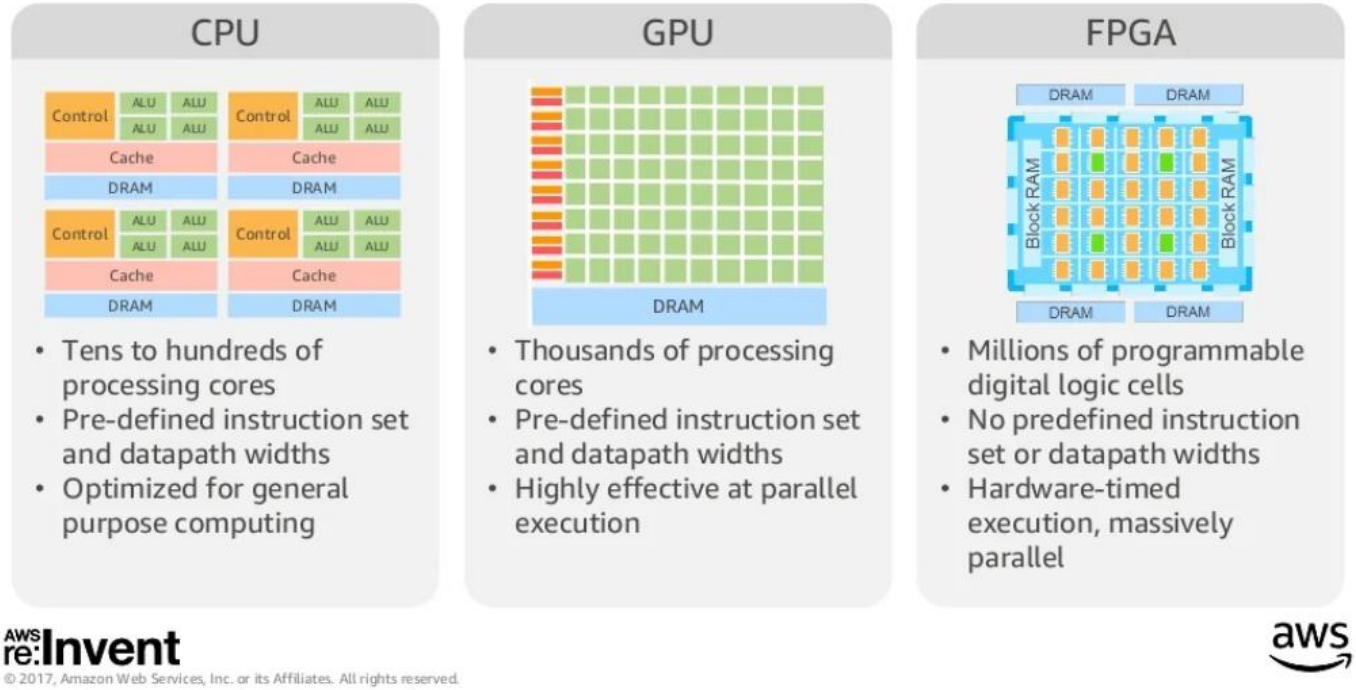
- Consta (principalmente) de:
  - CLB: Bloques lógicos configurables formados por componentes combinacionales y secuenciales
  - IOB: Bloques de entrada/salida
  - BRAM: Bloques de memoria RAM
  - Enrutado: Cables que transmiten señales
  - MMCMs, uP cores, FEC blocks, etc…



DENTRO DE UNA FPGA…



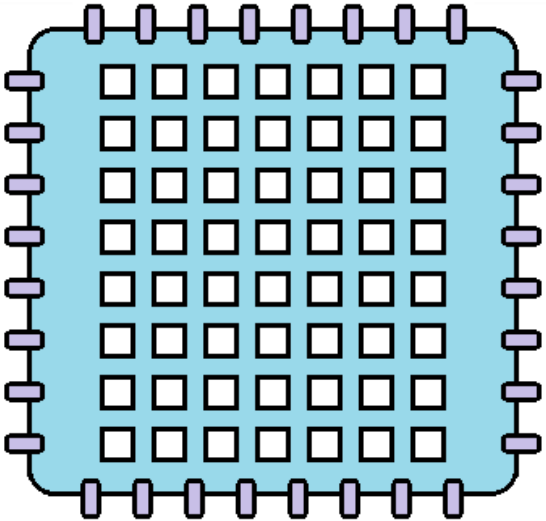
DIFERENTES OPCIONES



Para una CPU, se desarrolla software que se adecúe al hardware, lo que puede implicar muchas limitaciones. En una FPGA, se desarrolla hardware que se adecúe al software.

FPGA vs ASIC

- Un ASIC (Application Specific Integrated Circuit) es un dispositivo semiconductor fabricado para un propósito concreto



FPGA



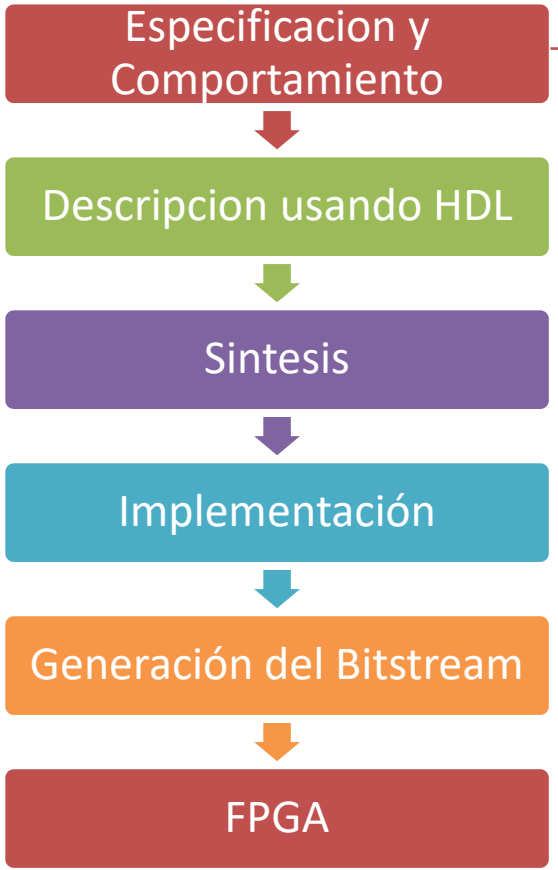
ASIC

FPGA vs ASIC

- Un ASIC (Application Specific Integrated Circuit) es un dispositivo semiconductor fabricado para un propósito concreto

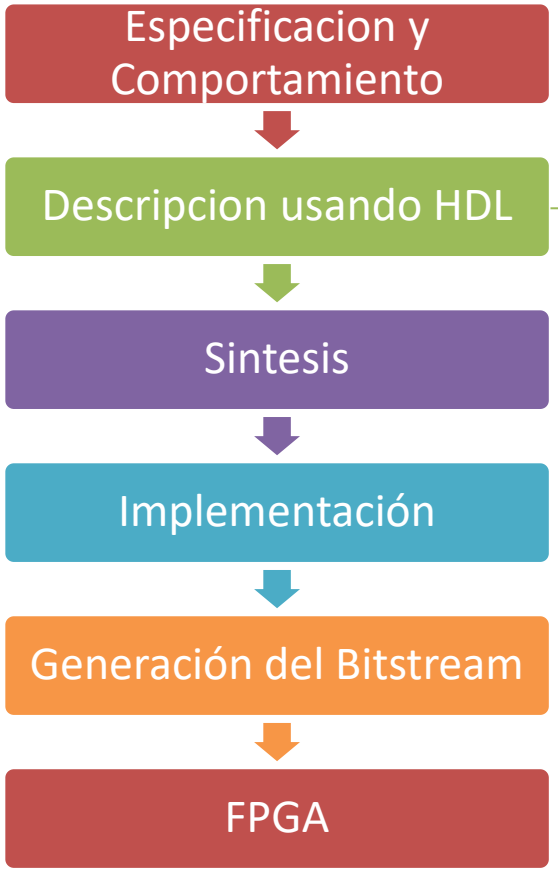
	FPGA	ASIC
Reprogramable	Sí	No
Fabricación	Rápida / Sencilla	Lenta / Compleja
Coste unitario	Alto	Bajo
Consumo de energía	Alto	Bajo
Rendimiento	Medio	Alto
Utilización	Prototipado	Aplicaciones concretas

FLUJO DE DISEÑO HARDWARE



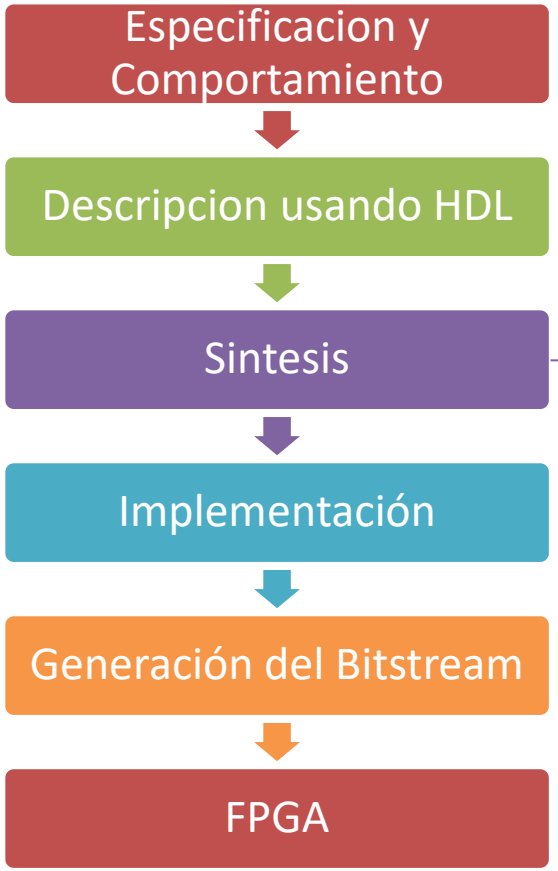
- Analisis de viabilidad.
- Division del algoritmo en bloques. funcionales.
- Analisis de precision finita.

FLUJO DE DISEÑO HARDWARE



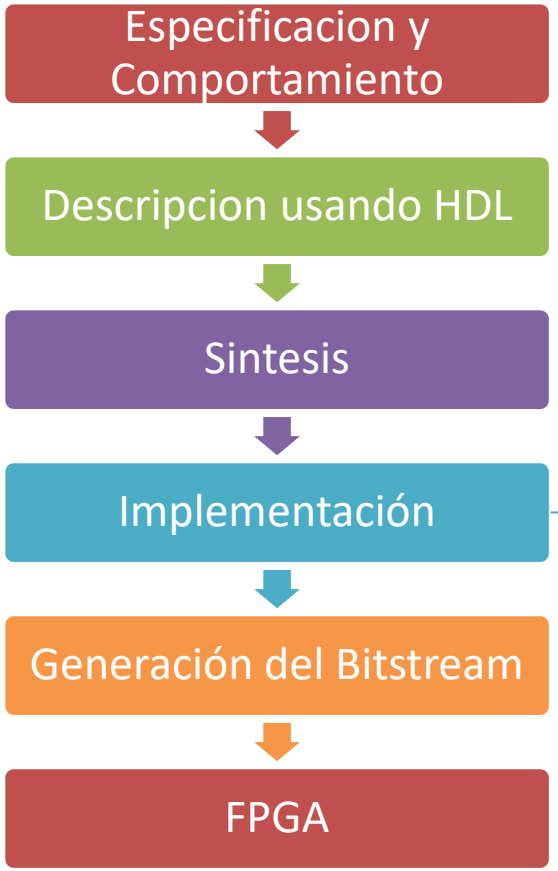
- Descripcion de cada uno de los bloques funcionales usando HDL
- Interconexion de los bloques

FLUJO DE DISEÑO HARDWARE



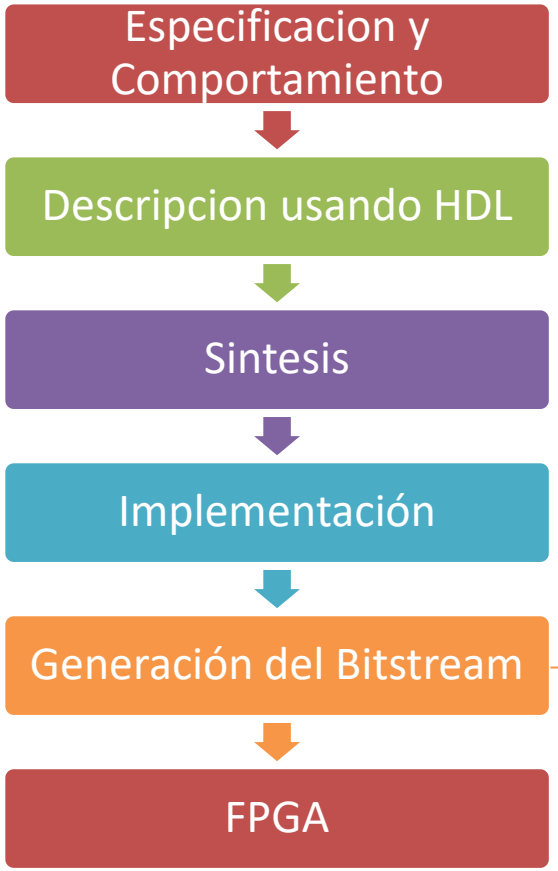
- Conversion del codigo HDL a una implementación basada en circuitos logicos.
- Usualmente llevada a cabo usando software que provee el fabricante de la FPGA.
- Optimización del diseño, eliminando logica redundante.

FLUJO DE DISEÑO HARDWARE



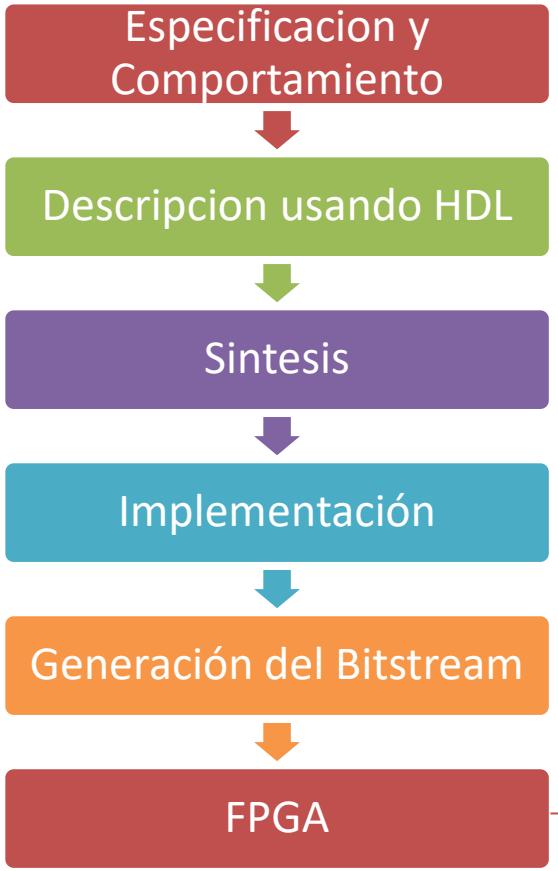
- Conversión de los elementos logicos a elementos primitivos disponibles en la FPGA.
- "Mapeo" de los elementos primitivos al layout de la FPGA, siguiendo las condiciones (*constraints)* indicadas por el diseñador.
- Interconexion de los elementos primitivos para cumplir con las condiciones de frecuencia de operación definidas por el diseñador.

FLUJO DE DISEÑO HARDWARE



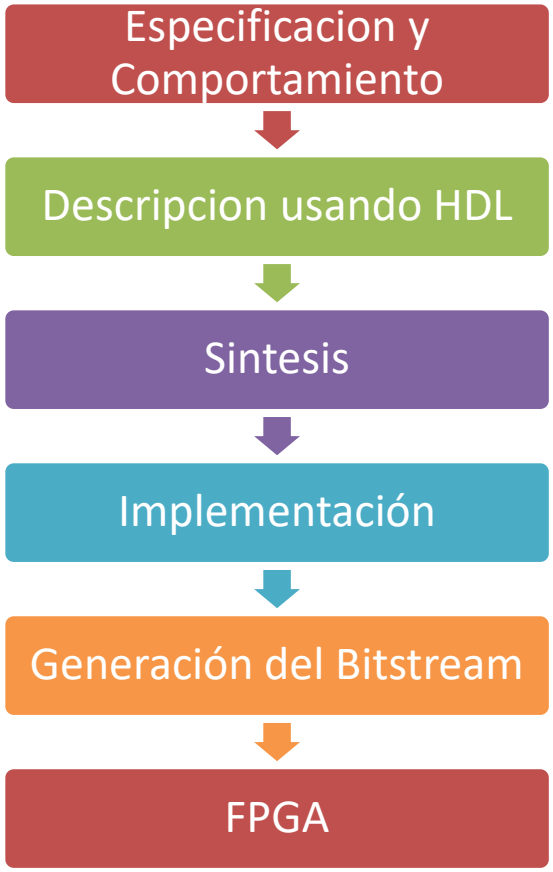
- Generacion de un archivo binario que contiene toda la información del diseño.
- El archivo binario permite configurar las conexiones en la FPGA.

FLUJO DE DISEÑO HARDWARE



- El bitstream se carga* en la memoria Flash de la FPGA para implementar el diseño.

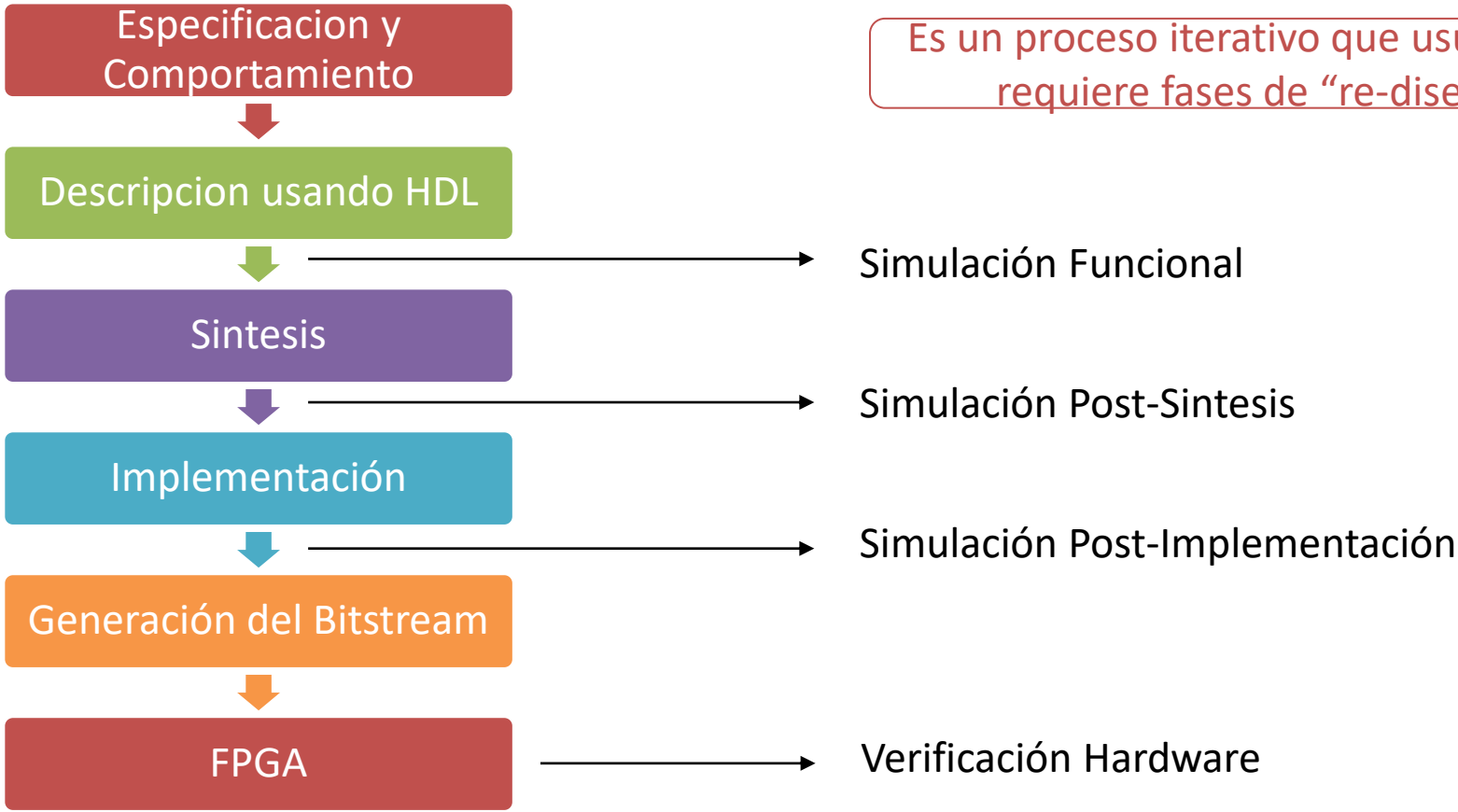
FLUJO DE DISEÑO HARDWARE



- Corre a cargo del diseñador.

- Se usa software especializado.
- El diseñador, configura el proceso y establece las *constraints* necesarias para cumplir con las condiciones de diseño.

FLUJO DE DISEÑO HARDWARE



Es un proceso iterativo que usualmente requiere fases de “re-diseño”.

FUNDAMENTOS DE VHDL

- Fin de línea: las instrucciones acaban con punto y coma ;
- Sensibilidad: VHDL no es sensible ni a mayúsculas ni a espacios en blanco
  - `S1 <= A and B;` → `s1 <= a AnD B ;`
- Comentarios: empiezan con dos guiones
  - `-- Esto es un comentario en VHDL`
- Paréntesis: se ponen por claridad, las operaciones tienen prioridad establecida

```
if x = '0' and y = '0' or z = '1' then
    blah; -- some useful statements
end if;

if ( ((x = '0') and (y = '0')) or (z = '1') ) then
    blah; -- some useful statements
end if;
```

Mealy, B. y Tappero, F. (2013), Free Range VHDL

FUNDAMENTOS DE VHDL

- Identificadores: se refiere al nombre dado a señales, puertos, variables en VHDL.
  - Identificadores validos:
    - `data_bus  -- descriptive name`
    - `WE  -- classic write enable`
    - `div_flag  -- good one`
    - `in_bus  -- input bus`
    - `clk  -- classic clock signal`
    - `clk_in`
    - `clk_out`
    - `mem_read_data`

Mealy, B. y Tappero, F. (2013), Free Range VHDL

FUNDAMENTOS DE VHDL

- Identificadores: se refiere al nombre dado a señales, puertos, variables en VHDL.
  - Identificadores NO validos:
    - `3Bus_val   -- begins with a number`
    - `DDD        -- not self commenting`
    - `mid_$num   -- illegal character`
    - `last__val  -- consec. underscores`
    - `str_val_   -- end with underscore`
    - `in         -- uses VHDL reserved word`
    - `#@$%%      -- total garbage`
    - `it_sucks   -- try to avoid`
    - `Big_vAlUe  -- valid but ugly`
    - `sim-val    -- illegal character (dash)`

Mealy, B. y Tappero, F. (2013), Free Range VHDL

FUNDAMENTOS DE VHDL

- Palabras reservadas: no pueden ser usadas como identificadores en VHDL. La siguiente lista incluye ALGUNAS de estas palabras reservadas.

A short list of VHDL reserved words

access	after	alias	all	attribute	block
body	buffer	bus	constant	exit	file
for	function	generic	group	in	is
label	loop	mod	new	next	null
of	on	open	out	range	rem
return	signal	shared	then	to	type
until	use	variable	wait	while	with

Mealy, B. y Tappero, F. (2013), Free Range VHDL

FUNDAMENTOS DE VHDL

Representacion de Caracteres:

- Caracteres: se definen entre comillas simples `'0'` o `'1'`
- Cadenas de caracteres: se definen con comillas dobles `"1010"`
- Cadenas de bits: se escribe la base seguida de la cadena de caracteres
  - Binario (B): `B"1010"`
  - Hexadecimal (X): `X"FA0"`
  - Octal (O): `O"372"`
  - Decimal (D): `D"23"`

¡Ojo! Si se omite la base se considera que es una cadena en binario `"1010"`

FUNDAMENTOS DE VHDL

Tipos de datos `STD_LOGIC` Y `STD_LOGIC_VECTOR`

- Son un estándar industrial y los utilizaremos SIEMPRE para definir puertos

- `STD_LOGIC` : valor presente en un *cable* de **1 bit**
- `STD_LOGIC_VECTOR`: para definir buses de datos (**vector de bits**)

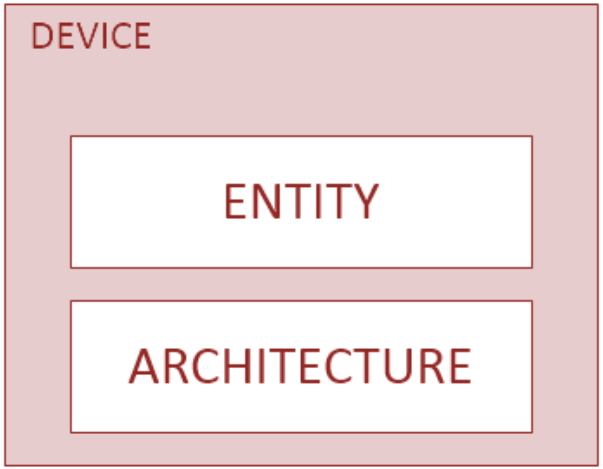
- Valores que pueden tomar:
- `'0'` : Nivel lógico bajo
- `'1'` : Nivel lógico alto
- `'U'` : Valor no definido, debido a que la señal no está inicializada
- `'X'` : Valor desconocido, debido a un cortocircuito
- `'Z'` : Alta impedancia, utilizado para "desconectar" señales

FUNDAMENTOS DE VHDL

Pregunta
teoría a pillar

Estructura del codigo en VHDL

- Los diseños hardware en VHDL constan de:
  - Librerías: se añaden en el encabezado para incluir otros librerías, *packages*, funciones, otros circuitos, etc.
  - Entidad: describe el circuito como una **caja negra** con sus puertos de entrada/salida
  - Arquitectura: describe el contenido de la caja negra y su comportamiento



ESTRUCTURA DE CODIGO VHDL

Ejemplo: Multiplexor 2-1:

```
-- STANDARD LIBRARIES
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

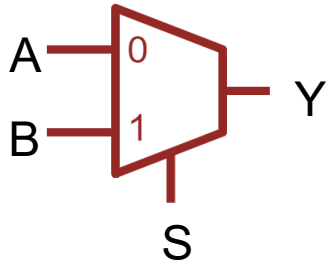
-- ENTITY DEFINITION
entity Mux2_1 is
      port( A : in   STD_LOGIC;
            B : in   STD_LOGIC;
            S : in   STD_LOGIC;
            Y : out  STD_LOGIC
      );
end Mux2_1;

-- ARCHITECTURE DEFINITION
architecture Behavior of Mux2_1 is
begin

      Y <= A when (S = '0') else B;

end Behavior;
```

Librerias necesarias para poder usar los tipos `STD_LOGIC` y `STD_LOGIC_VECTOR`

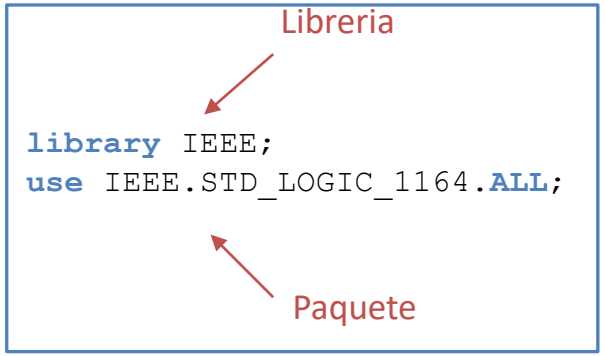


S	Y
0	A
1	B

ESTRUCTURA DE CODIGO VHDL

Librerias:

- Una librería es un conjunto de paquetes
- Debemos especificar los paquetes que usamos de la librería, no vale con incluir sólo la librería



- Los paquetes más comunes de la librería IEEE son:
  - `STD_LOGIC_1164`: Estándar para definir señales y operaciones lógicas
  - `NUMERIC_STD`: Para definir números `signed` y `unsigned` y operaciones aritméticas
  - `STD_LOGIC_TEXTIO`: Para trabajar con ficheros y cadenas de caracteres

- Podemos crear nuestros propios paquetes y librerías!

ESTRUCTURA DE CODIGO VHDL

Pregunta
teoría a pillar

ENTIDAD:

- La entidad describe cómo se conecta el componente a otros diseños
- Define el componente como “caja negra”
- Una entidad puede contener los siguientes campos:
  - Ports: son los puertos de entrada/salida que permiten que el componente se conecte con el exterior.
  - Generics: permite definir parámetros del componente de modo que no sea necesario repetir su definición una y otra vez.
  - Constants: permite definir constantes que se utilizarán en la arquitectura del componente.

ESTRUCTURA DE CODIGO VHDL

ENTIDAD:

```
entity ejemplo is
    port( in1    : in  std_logic;
          in2    : in  std_logic;
          out1   : out std_logic
    );
    generic( gen1 : integer := 4;
             gen2 : time := 10 ns
    );
end ejemplo;
```

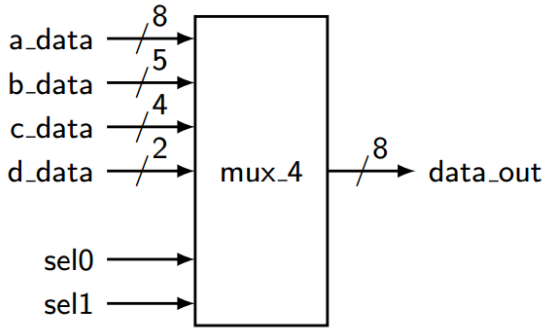
2 puertos de entrada y uno de salida

Los *generic* son como los #define en C

Una entidad puede contener los 2 campos… ¡o ninguno!

ESTRUCTURA DE CODIGO VHDL

- Podemos definir puertos en una entidad que “agrupen” mas de un bit. Para ello usamos **std_logic_vector**.




```
entity mux_4 is
     port( a_data : in std_logic_vector(7 downto 0);
           b_data : in std_logic_vector(4 downto 0);
           c_data : in std_logic_vector(3 downto 0);
           b_data : in std_logic_vector(1 downto 0);
           sel0, sel1 : in std_logic
     );
end mux_4;
```

- Las señales en cada “bus” se pueden especificar en dos ordenes: **downto** y **to**.
  - **downto**: el bit mas significativo esta a la izquierda del bus. Es comunmente usado cuando el bus representa un numero.
  - **to**: es comunmente utilizado cuando el bus representa un arreglo de bits.

ESTRUCTURA DE CODIGO VHDL

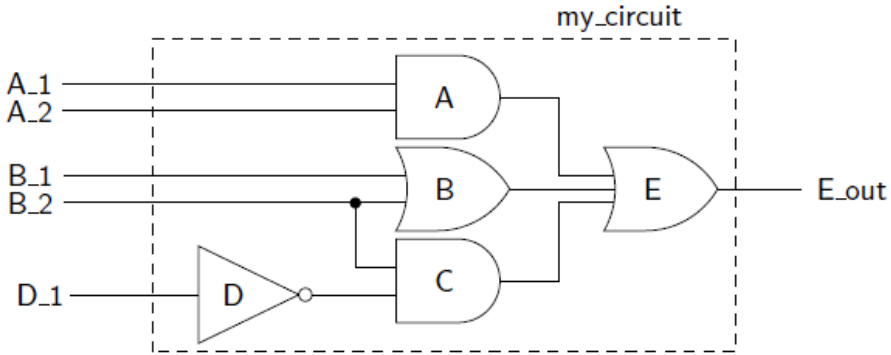
ENTIDAD (TIPOS DE DATOS):

- **`In:`** define un puerto de entrada de datos. El valor de la señal:
  - Puede ser leído dentro de la entidad
  - No puede ser modificado dentro de la entidad
- **`Out:`** define un puerto de salida de datos. El valor de la señal:
  - No puede ser leído dentro de la entidad
  - Puede ser actualizado dentro de la entidad que lo define
- **`Inout:`** define un puerto que puede ser entrada y salida de datos. El valor de la señal:
  - Puede ser leído dentro de la entidad
  - Puede ser actualizado dentro de la entidad que lo define
- **`Buffer:`** define un puerto como el tipo inout pero:
  - Sólo se puede conectar a otra señal de tipo buffer
  - No puede tener más de un driver (más de una entidad que modifique su valor)

ESTRUCTURA DE CODIGO VHDL

ARCHITECTURE:

- La arquitectura describe el comportamiento del componente
- En la arquitectura se ejecutan las operaciones en paralelo



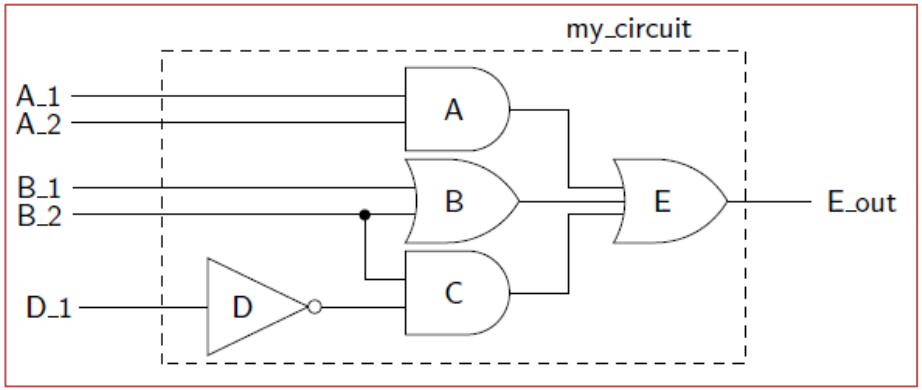
La entidad define la “caja negra” y la arquitectura el “contenido”

ESTRUCTURA DE CODIGO VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity my_circuit is
     port( A_1 : in    STD_LOGIC;
           A_2 : in    STD_LOGIC;
           B_1 : in    STD_LOGIC;
           B_2 : in    STD_LOGIC;
           D_1 : in    STD_LOGIC;
           E_out : out  STD_LOGIC);
end my_circuit;

architecture behavior of my_circuit is
     signal A_out, B_out, C_out : std_logic;
begin
     A_out <= A_1 and A_2;
     B_out <= B_1 or B_2;
     C_out <= (not D_1) and B_2;
     E_out <= A_out or B_out or C_out;
end behavior;
```

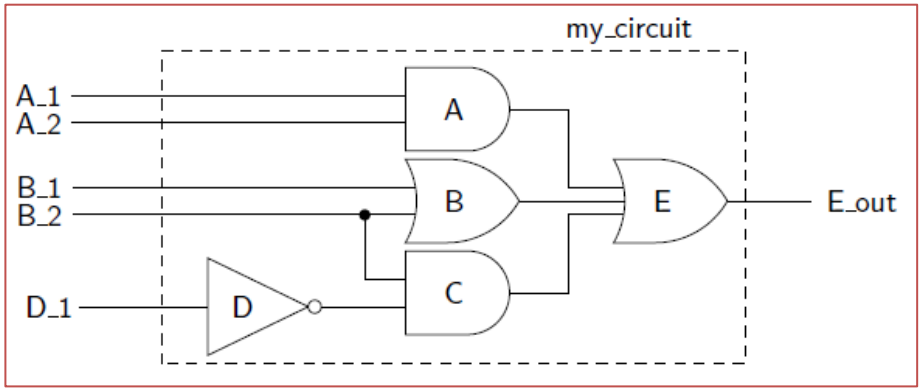


ESTRUCTURA DE CODIGO VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity my_circuit is
     port( A_1 : in    STD_LOGIC;
           A_2 : in    STD_LOGIC;
           B_1 : in    STD_LOGIC;
           B_2 : in    STD_LOGIC;
           D_1 : in    STD_LOGIC;
           E_out : out  STD_LOGIC);
end my_circuit;

architecture behavior of my_circuit is
     signal A_out, B_out, C_out : std_logic;
begin
     A_out <= A_1 and A_2;
     B_out <= B_1 or B_2;
     C_out <= (not D_1) and B_2;
     E_out <= A_out or B_out or C_out;
end behavior;
```



Las señales, constantes, componentes, tipos de datos, se declaran antes del `begin`.

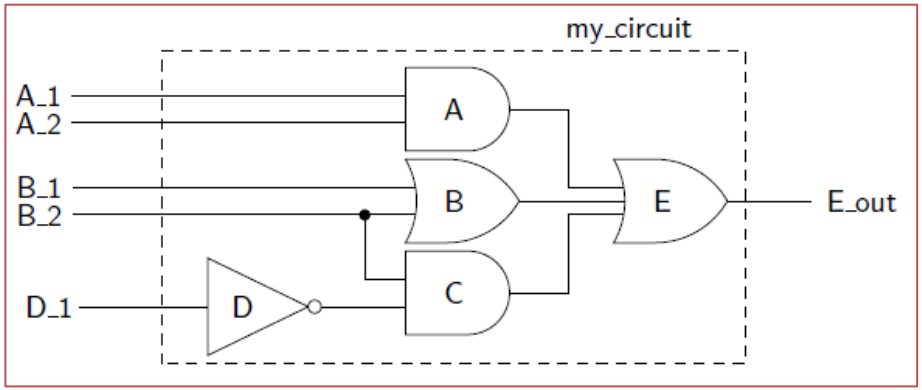
`signal` es la representación software de “un cable”.

ESTRUCTURA DE CODIGO VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity my_circuit is
     port( A_1 : in    STD_LOGIC;
           A_2 : in    STD_LOGIC;
           B_1 : in    STD_LOGIC;
           B_2 : in    STD_LOGIC;
           D_1 : in    STD_LOGIC;
           E_out : out  STD_LOGIC);
end my_circuit;

architecture behavior of my_circuit is
     signal A_out, B_out, C_out : std_logic;
begin
     A_out <= A_1 and A_2;
     B_out <= B_1 or B_2;
     C_out <= (not D_1) and B_2;
     E_out <= A_out or B_out or C_out;
end behavior;
```



- El operador <= se usa como “asignación de señales”.
- Note que las operaciones se ejecutan de forma concurrente (en paralelo).
- El valor de E_out se calcula un tiempo despues (?) que el resto de operaciones.

ESTRUCTURA DE CODIGO VHDL



SIMULACION

ESTRUCTURA DE CODIGO VHDL

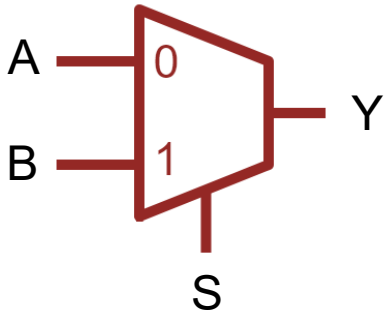
ASIGNACION CONDICIONAL (**when**)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity Mux2_1 is
     port( A : in   STD_LOGIC;
           B : in   STD_LOGIC;
           S : in   STD_LOGIC;
           Y : out STD_LOGIC
     );
end Mux2_1;

architecture Behavior of Mux2_1 is
begin

     Y <= A when (S = '0') else B;

end Behavior;
```



S	Y
0	A
1	B

ESTRUCTURA DE CODIGO VHDL

ASIGNACION CONDICIONAL (**when**)

- Forma general:

```
<target> <=   <expression> when <condition> else
              <expression> when <condition> else
              <expression>;
```

ESTRUCTURA DE CODIGO VHDL

Ejemplo: escribe la arquitectura VHDL que implemente la función $F3 = \overline{LM}N + LM$. Use solo asignaciones condicionales en su solución.

SOLUCION:

$$F3 = \overline{LM}N + LM$$

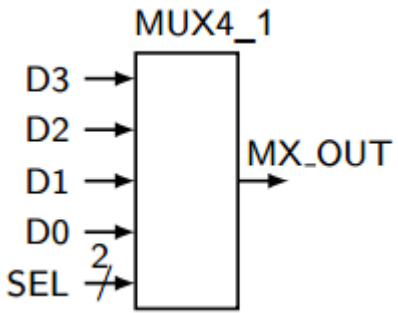
```
F3 <=  '1' when L='0' and M='0' and N='1' else
       '1' when L='1' and M='1' else
       '0';
```

ESTRUCTURA DE CODIGO VHDL

EJEMPLO: Escribe el codigo VHDL que implemente un MUX 4:1, usando una asignación condicional. Las entradas al multiplexor son las entradas de datos `D0`, `D1`, `D2`, `D3` y de control `SEL` (de dos bits). La salida es `MX_OUT`.

Además existe manera con Case o con Select

SOLUCION:




```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX4_1 is
      port( D0, D1, D2, D3 : in STD_LOGIC;
            SEL : in STD_LOGIC_VECTOR(1 downto 0)
            MX_OUT : out STD_LOGIC );
end MUX4_1;

architecture mymux of MUX4_1 is
begin
      MX_OUT <=   D0 when SEL = "00" else
                  D1 when SEL = "01" else
                  D2 when SEL = "10" else
                  D3 when SEL = "11" else
                  '0';
end mymux;
```

ESTRUCTURA DE CODIGO VHDL

PROCESS

- Es una contrucción concurrente, que nos permite ejecutar instrucciones de forma secuencial
- En el *process*, las instrucciones se ejecutan una tras otra de arriba abajo
- Fuera del *process* las instrucciones se siguen ejecutando en paralelo
- Si hay varios *process*, se ejecutan en paralelo entre ellos

```
-- Ejemplo de process
process (lista_de_sensibilidad)
    -- declaración de variables
begin
    -- instrucciones secuenciales
end process;
```

El `process` se ejecuta cuando alguno de los valores de la lista de sensibilidad cambia

ESTRUCTURA DE CODIGO VHDL

PROCESS

XOR usando asignación concurrente

```
-- library declaration
library IEEE;
use IEEE.std_logic_1164.all;

-- entity
entity my_xor is
   port ( A,B : in std_logic;
          F : out std_logic);
end my_xor;

-- architecture
architecture dataflow of my_xor is
begin

   F <= A XOR B;

end dataflow;
```

XOR usando process

```
-- library declaration
library IEEE;
use IEEE.std_logic_1164.all;

-- entity
entity my_xor is
   port ( A,B : in std_logic;
          F : out std_logic);
end my_xor;

-- architecture
architecture behav of my_xor is
begin

xor_proc: process(A,B) is
begin
   F <= A XOR B;
end process xor_proc;

end behav;
```

ESTRUCTURA DE CODIGO VHDL

PROCESS

- Dentro de un process, no solo se utilizan asignaciones concurrentes, Tambien se utilizan estructuras `if-else`, `case` and `for`.

```
if (condition) then
     <statements>
elsif (condition) then
     <statements>
else
     <statements>
end if;
```

$$F_OUT(A,B,C) = A\overline{BC}+BC$$

```
proc1: process(A,B,C) is
begin
   if (A = '1' and B = '0' and C = '0') then
      F_OUT <= '1';
   elsif (B = '1' and C = '1') then
      F_OUT <= '1';
   else
      F_OUT <= '0';
   end if;
end process proc1;
```

ESTRUCTURA DE CODIGO VHDL

PROCESS

- Dentro de un process, no solo se utilizan asignaciones concurrentes, Tambien se utilizan estructuras `if-else`, `case` and `for`.

```
case (expression) is
   when choices =>
      <sequential statements>
   when choices =>
      <sequential statements>
   when others =>
      <sequential statements>
end case;
```

$$F_OUT(A,B,C) = A\overline{BC}+BC$$

```
architecture my_sol of my_example is
   signal ABC: std_logic_vector(2 downto 0);
begin

ABC <= A & B & C; -- group signals for case statement

my_proc: process (ABC)
begin
   case (ABC) is
      when "100" => F_OUT <= '1';
      when "011" => F_OUT <= '1';
      when "111" => F_OUT <= '1';
      when others => F_OUT <= '0';
   end case;
end process my_proc;

end my_sol;
```

ESTRUCTURA DE CODIGO VHDL

PROCESS

- Dentro de un process, no solo se utilizan asignaciones concurrentes, Tambien se utilizan estructuras `if-else`, `case` and `for`.

```
-- for loop
my_label: for index in a_range loop
   sequential statements...
end loop my_label;
```

```
architecture my_sol of my_example is
   signal A, B : std_logic_vector(7 downto 0);
begin

my_proc: process (A, B)
begin
   for ii in 0 to 7 loop
      if ii < 3 then
            C(ii) <= A(ii) and B(ii);
      else
            C(ii) <= A(ii) or B(ii);
      end if;
   end loop;
end process my_proc;

end my_sol;
```

TIPOS DE VARIABLES EN VHDL

- Constantes: se utilizan cuando se necesita un valor estático.
  Ej: inicializar registros con un valor predefinido o para comparar señales con un valor constante
  - `constant a : integer := 1; constant b : real := 0.123;`
  - `constant c : std_logic_vector(2 downto 0) := "010";`

- Señales: se utilizan para definir conexiones externas (cables) del diseño.
  Ej: entradas/salidas de un módulo que se conectan con otros módulos
  - `signal sig1 : integer;`
  - `signal sig2 : std_logic := '0';`

```
sig1 <= a;
sig2 <= c(1);
```

Inicializacion de señales

- Variables: son los valores internos dentro de un process
  - `variable var1 : integer;`
  - `variable var2 : integer := 1;`

```
var1 := var2;
```

Asignación de variables

OPERADORES EN VHDL

Highest priority
(evaluated first)

Unary miscellaneous operator	**abs not and or nand nor xor xnor**
Binary miscellaneous operator	**
Multiplying operator	* / mod rem
Sign	+ –
Adding operator	+ – &
Shift operator	sll srl sla sra rol ror
Relational operator	= /= < <= > >= ?= ?/= ?< ?<= ?> ?>=
Logical operator	and or nand nor xor xnor
Condition operator	??

- El orden en que se presenta la tabla, indica el orden de la precedencia de la operación.
- Aunque no es necesario, para facilitar la “lectura” y comprensión del codigo, es una buena practica el uso de parentesis para separar operaciones.

https://vhdlwhiz.com/operator-precedence-in-vhdl/

OPERADORES EN VHDL

- Shift Operations:

Operator	Name		Example	Result
logical	sll	shift left logical	result <= "10010101" sll 2	"01010100"
	srl	shift right logical	result <= "10010101" srl 3	"00010010"
arithmetic	sla	shift left arithmetic	result <= "10010101" sla 3	"10101111"
	sra	shift right arithmetic	result <= "10010101" sra 2	"11100101"
rotate	rol	rotate left	result <= "101000" rol 2	"100010"
	ror	rotate right	result <= "101001" ror 2	"011010"

- Resto de Operaciones:

Operator		Name	Comment
addition	+	addition	
	–	subtraction	
unary	+	identity	
	–	negation	
multiplying	*	multiplication	
	/	division	often limited to powers of two
	mod	modulus	can operate only on specific types
	rem	remainder	can operate only on specific types
other	**	exponentiation	often limited to powers of two
	abs	absolute value	
	&	concatenation	can operate only on specific types

OPERADORES EN VHDL

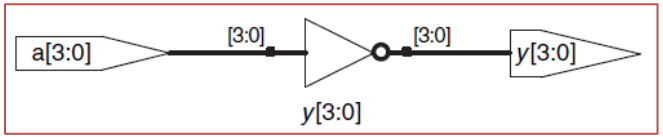
- Concatenacion: es una “operacion” muy utilizada en VHDL. Permite agrupar señales en un unico bus.

```
signal A_val, B_val : std_logic_vector(3 downto 0);
signal C_val : std_logic_vector(5 downto 0);
signal D_val : std_logic_vector(7 downto 0);
------
C_val <= A_val & "00";
C_val <= "11" & B_val;
C_val <= '1' & A_val & '0';
D_val <= "0001" & C_val(3 downto 0);
D_val <= A_val & B_val;
```

CIRCUITOS COMBINACIONALES CON VHDL

INVERSOR

Ejemplo: inversor de 4 bits




```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity inversor is
   port( A : in std_logic_vector(3 downto 0);
         Y : out std_logic_vector(3 downto 0)
   );
end inversor;

architecture Behavior of inversor is
begin
    Y <= not A;
end Behavior;
```

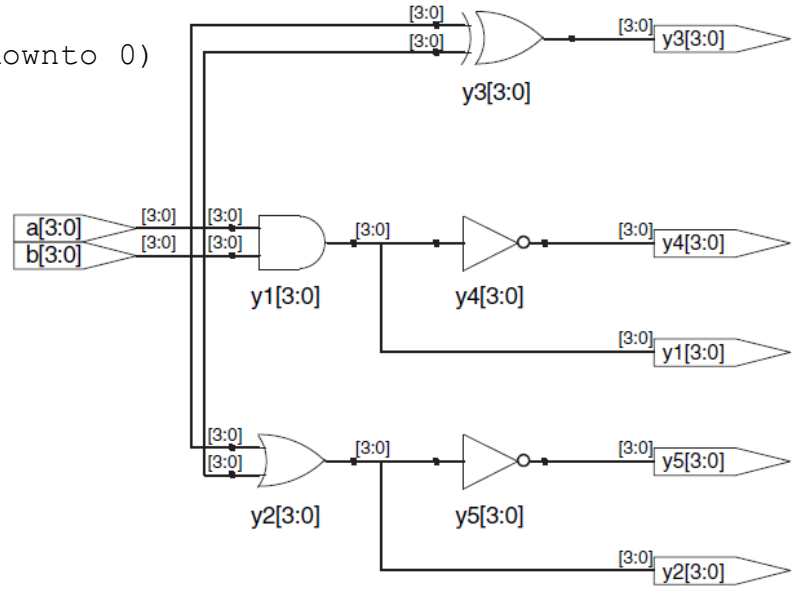
CIRCUITOS COMBINACIONALES CON VHDL

PUERTAS LOGICAS

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

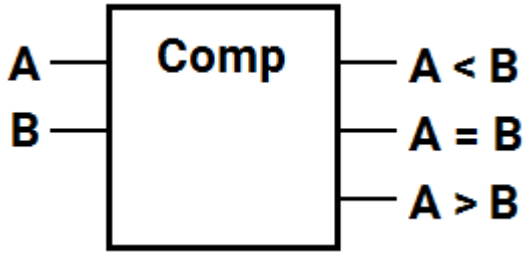
entity gates is
   port( A,B : in std_logic_vector(3 downto 0);
         Y1,Y2,Y3,Y4,Y5 : out std_logic_vector(3 downto 0)
   );
end gates;

architecture Behavior of gates is
begin
     Y1 <= A and B;
     Y2 <= A or B;
     Y3 <= A xor B;
     Y4 <= A nand B;
     Y5 <= A nor B;
end Behavior;
```



CIRCUITOS COMBINACIONALES CON VHDL

COMPARADOR DE 8 BITS




```
entity comparator is
   port( A,B : in std_logic_vector(7 downto 0);
         ALB, AGB, AEB : out std_logic
   );
end comparator;

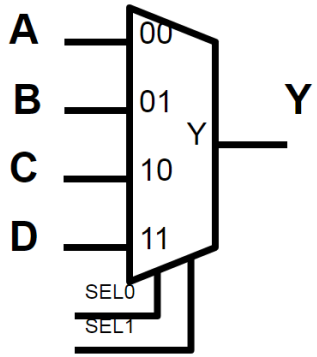

architecture Behavior of comparator is
begin
   process (A, B)
   begin

   end process;
end Behavior;
```

```
if (A < B) then
     ALB <= '1';
     AGB <= '0';
     AEB <= '0';
elsif (A > B) then
     ALB <= '0';
     AGB <= '1';
     AEB <= '0';
else
     ALB <= '0';
     AGB <= '0';
     AEB <= '1';
end if;
```

CIRCUITOS COMBINACIONALES CON VHDL

MULTIPLEXOR 4:1




```
entity mux4_1 is
   port( A,B,C,D : in std_logic;
         SEL : in std_logic_vector(1 downto 0);
         Y : out std_logic
   );
end mux4_1;

architecture Behavior of mux4_1 is
begin
   process (A, B, C, D, SEL)
   begin

   end process;
end Behavior;
```

```
case SEL is
    when "00" =>
     Y <= A;
    when "01" =>
     Y <= B;
    when "10" =>
     Y <= C;
    when "11" =>
     Y <= D;
    when others =>
     Y <= '0';
end case;
```

CIRCUITOS SECUENCIALES CON VHDL

- Hasta ahora hemos visto como usar VHDL para describir circuitos combinacionales → NO tienen memoria
- En un circuito secuencial, las salidas del mismo dependen de las entradas actuales y de las secuencias de entradas anteriores → SI tienen memoria
- Para conseguir que tengan memoria se usa la realimentación de señales o bien bloques secuenciales como los latches y flip-flops

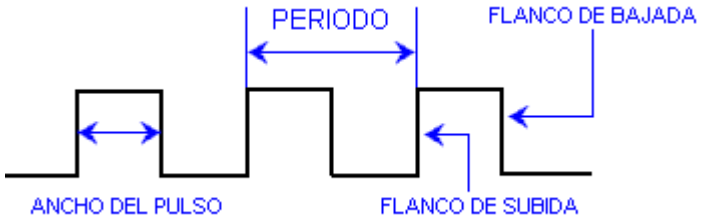
En el diseño hardware para FPGAs se suele evitar el uso de latch!

CIRCUITOS SECUENCIALES CON VHDL

BIESTABLES:

- Son componentes con dos estados usados para almacenar información binaria
- Latch: biestable asíncrono que muestrea las entradas continuamente y cambia las salidas en cualquier momento. No tiene señal de reloj

- Flip-flop: biestable síncrono que muestrea las entradas y cambia las salidas en ciertos instantes de tiempo marcados por una **señal de reloj**

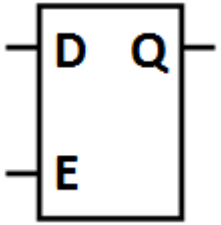


¿Cómo diseñamos un latch/flip-flop en VHDL?

¡USAMOS LOS *PROCESS*!

CIRCUITOS SECUENCIALES CON VHDL

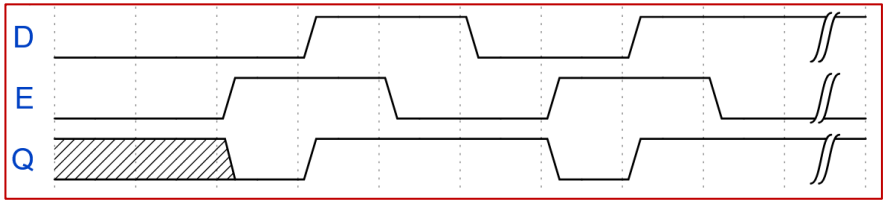
- El latch D mantiene el valor almacenado (estado) cuando la señal de `enable=0`, y deja fluir el dato cuando `enable=1`




```
entity latchD is
   port( D : in std_logic;
         E : in std_logic;
         Q : out std_logic
   );
end latchD;

architecture Behavior of latchD is
begin

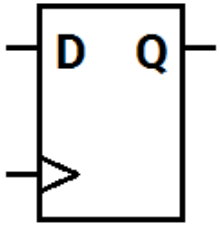
end Behavior;
```




```
process (D, E) begin
     if (E = '1') then
          Q <= D;
     end if;
end process;
```

CIRCUITOS SECUENCIALES CON VHDL

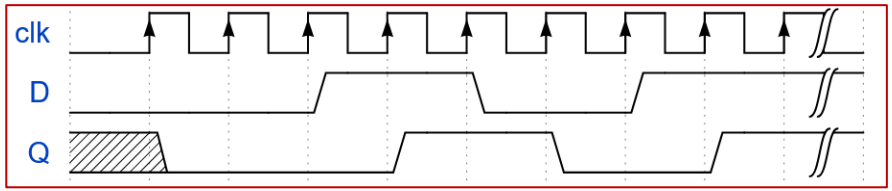
- El Flip-flop Tipo D actualiza el valor almacenado (estado) cuando llega un flanco (positivo / negativo) en la señal de reloj.




```
entity FlipFlopD is
   port( D : in std_logic;
         clk : in std_logic;
         Q : out std_logic
   );
end FlipFlopD;

architecture Behavior of FlipFlopD is
begin

end Behavior;
```




```
process (clk) begin
     if rising_edge(clk) then
          Q <= D;
     end if;
end process;
```

CIRCUITOS SECUENCIALES CON VHDL

Podemos tener registros de mas de un bit.

Reset sincrono activo a nivel alto. ¿Por que?

```
entity registro is
    port( CLK, RST : in std_logic;
          D : in std_logic_vector(7 downto 0);
          Q : out std_logic_vector(7 downto 0)
    );
end registro;
```

es lo mismo que
Q <= "00000000";

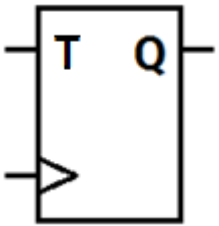
```
architecture Behavior of registro is
begin

end Behavior;
```

```
process (CLK) begin
   if (rising_edge(CLK)) then
      if (RST = '1') then
         Q <= (others => '0');
      else
         Q <= D;
      end if;
   end if;
end process;
```

CIRCUITOS SECUENCIALES CON VHDL

- Ejemplo: escribe el codigo VHDL que implemente un flip-flop tipo T. Considere la entrada S (active-low) que coloca a ‘1’ la salida del flip-flop cuando es activada.



T	Q	$Q_{siguiente}$
0	0	0
0	1	1
1	0	1
1	1	0

Si T = 1 cambia de estado, si T = 0 no cambia de estado

Q* = T xor Q

CIRCUITOS SECUENCIALES CON VHDL

- Ejemplo: escribe el codigo VHDL que implemente un flip-flop tipo T. Considere la entrada S (active-low) que coloca a ‘1’ la salida del flip-flop cuando es activada.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity t_ff_s is
   port ( T,S,CLK : in std_logic;
          Q : out std_logic);
end t_ff_s;

architecture my_t_ff_s of t_ff_s is

begin

tff: process (S,CLK)
    
end process tff;



end my_t_ff_s;
```

Error!!!

Una salida no puede ser leida

```
begin
if (S = '0') then
   Q <= '1';
elsif (rising_edge(CLK)) then
   Q <= T XOR Q;
end if;
```

CIRCUITOS SECUENCIALES CON VHDL

- Ejemplo: escribe el codigo VHDL que implemente un flip-flop tipo T. Considere la entrada S (active-low) que coloca a ‘1’ la salida del flip-flop cuando es activada.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity t_ff_s is
   port ( T,S,CLK : in std_logic;
          Q : out std_logic);
end t_ff_s;

architecture my_t_ff_s of t_ff_s is
    signal t_tmp : std_logic; -- intermediate signal declaration
begin

tff: process (S,CLK)
    
end process tff;

   Q <= t_tmp;

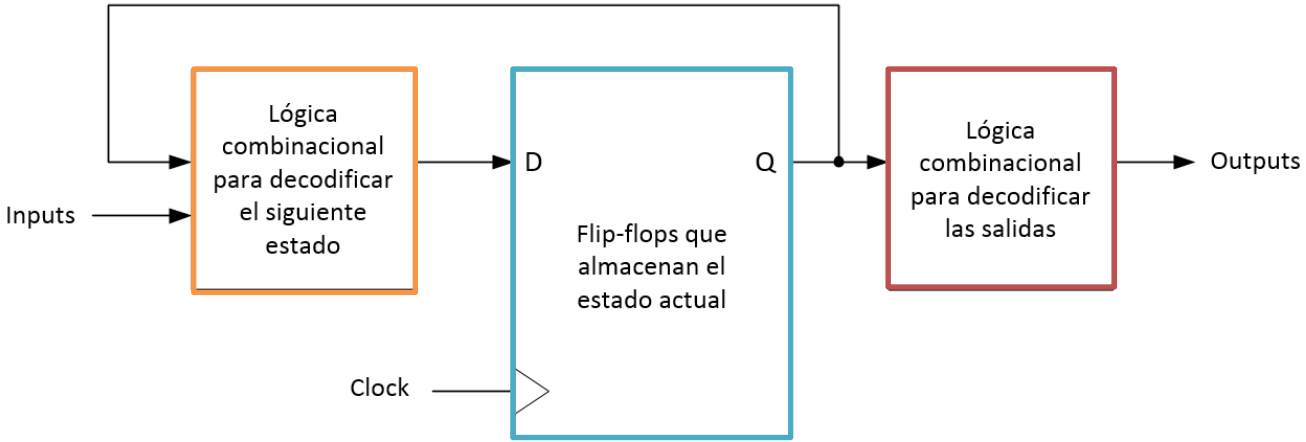
end my_t_ff_s;
```

```
begin
if (S = '0') then
   t_tmp <= '1';
elsif (rising_edge(CLK)) then
   t_tmp <= T XOR t_tmp; -- temp output assignment
end if;
```

MAQUINAS DE ESTADO

- Las máquinas de estado finito (FSMs) son abstracciones matemáticas utilizadas para resolver una gran variedad de problemas. Entre ellos:
  - Diseño de sistemas electrónicos automatizados
  - Diseño de protocolos de comunicación
  - Detectores o generadores de secuencias
  - …
- Hay (principalmente) máquinas de estado de Moore y de Mealy

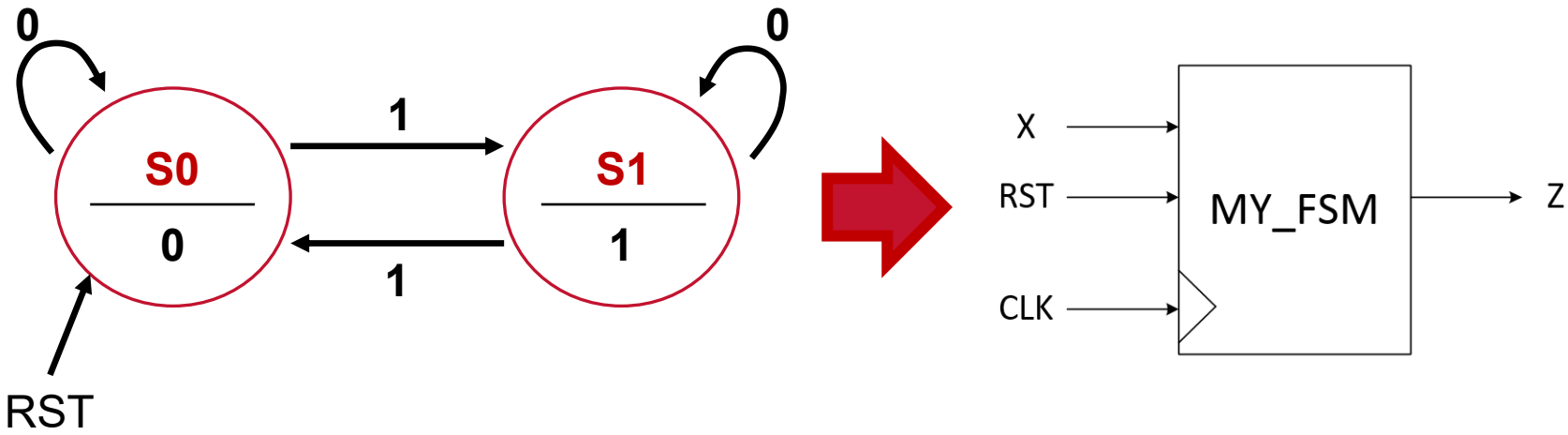
MAQUINAS DE ESTADO



- Decodificador del estado siguiente: utiliza el estado actual y el valor de las entradas para cambiar el estado de la máquina (ecuaciones de excitación)
- Registro de estados: conjunto de flip-flops que almacenan el estado actual
- Decodificador de salida: utilizado para generar la salida deseada

MAQUINA DE MOORE

- Dado el siguiente diagrama de estados de una máquina de Moore con una señal de entrada de un bit (X) y una señal de salida de un bit (Z), obtener su código VHDL.

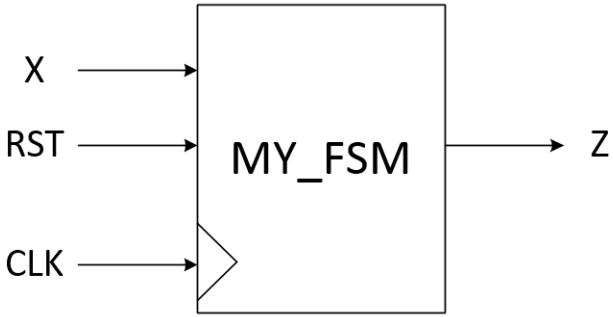


MAQUINA DE MOORE

- Empezamos definiendo la librería y la entidad:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity my_FSM is
    port(   CLK : in std_logic;
            RST : in std_logic;
            X : in std_logic;
            Z : out std_logic
    );
end my_FSM;
```

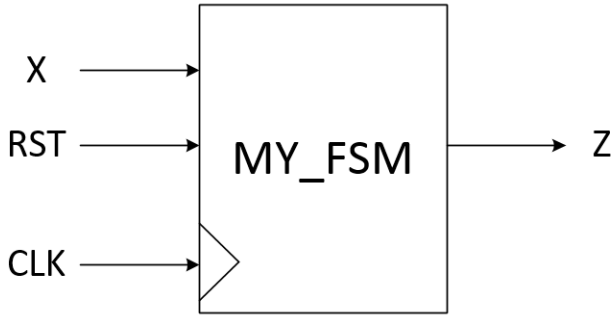


MAQUINA DE MOORE

- Empezamos definiendo la librería y la entidad:

```
architecture Behavior of my_FSM is
    -- Declaración de variables de estado
begin
    -- Proceso secuencial síncrono para
    -- controlar los flip-flop

    -- Proceso combinacional case-when para
    -- controlar los cambios de estado y la
    -- salida
end Behavior;
```




```
type state_type is (S0, S1);
signal current_state, next_state : state_type;
```

Se define un tipo de dato (enumerate) para definir los diferentes estados.

MAQUINA DE MOORE

```
architecture Behavior of my_FSM is
     type state_type is (S0, S1);
     signal current_state, next_state : state_type;
begin
     -- Proceso secuencial síncrono para
     -- controlar los flip-flop

     -- Proceso combinacional case-when para
     -- controlar los cambios de estado y la salida

end Behavior;
```

Utilizo **flip-flops D** para el cambio de estado

```
process(CLK) begin
   if (rising_edge(CLK)) then
      if (RST = '1') then
         current_state <= S0;
      else
         current_state <= next_state;
      end if;
   end if;
end process;
```

MAQUINA DE MOORE

```
architecture Behavior of my_FSM is
   type state_type is (S0, S1);
   signal current_state, next_state : state_type;
Begin

process(CLK) begin
   if (rising_edge(CLK)) then
      if (RST = '1') then
          current_state <= S0;
      else
          current_state <= next_state;
      end if;
   end if;
end process;

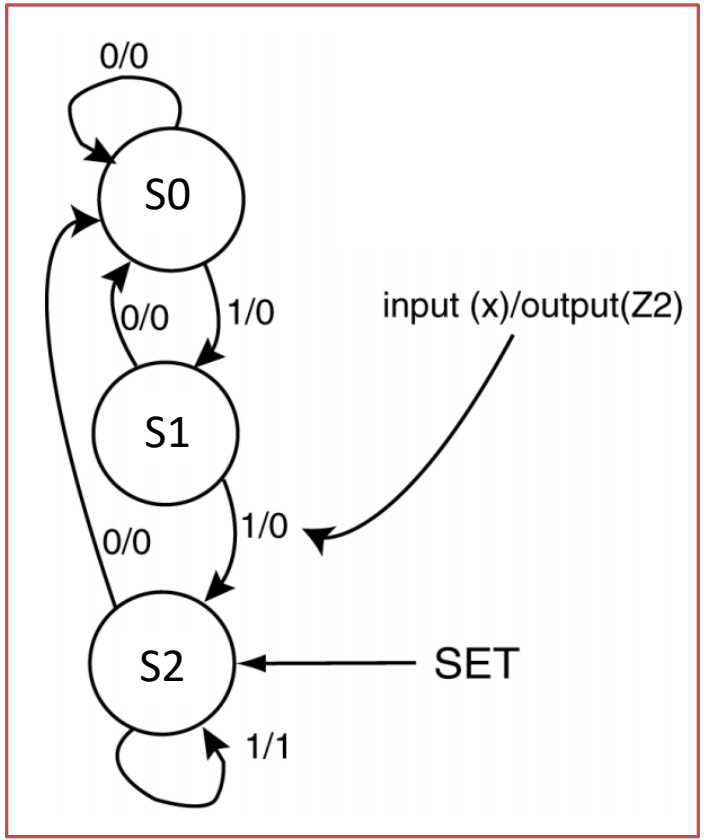
-- Proceso combinacional case-when para controlar
-- los cambios de estado y la salida

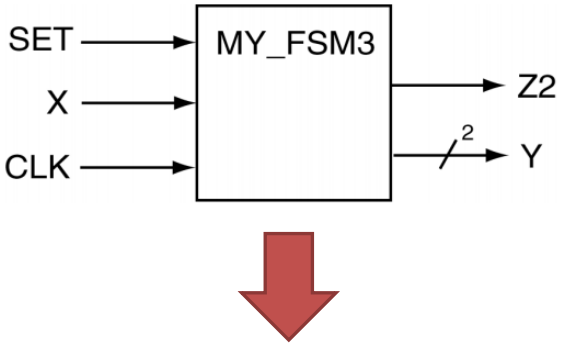
end Behavior;
```

```
process(current_state, X)
begin
   case current_state is
      when S0 =>
         Z <= '0'; -- Salida de Moore
         if (X = '1') then
            next_state <= S1;
         else
            next_state <= S0;
         end if;
      when S1 =>
         Z <= '1'; -- Salida de Moore
         if (X = '1') then
            next_state <= S0;
         else
            next_state <= S1;
         end if;
      when others =>
         Z <= '0';
         next_state <= S0;
   end case;
end process;
```

La(s) salida(s) solo dependen del estado actual y no de las entradas.

MAQUINA DE MEALY

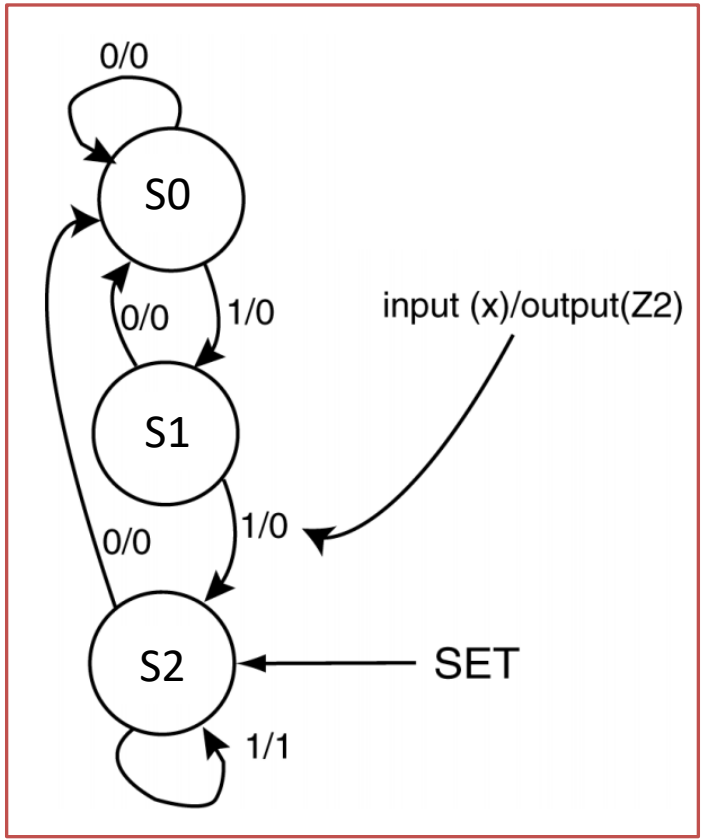





```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity my_FSM3 is
     port(    CLK : in std_logic;
              SET : in std_logic;
              X : in std_logic;
              Z2 : out std_logic
     );
end my_FSM3;
```

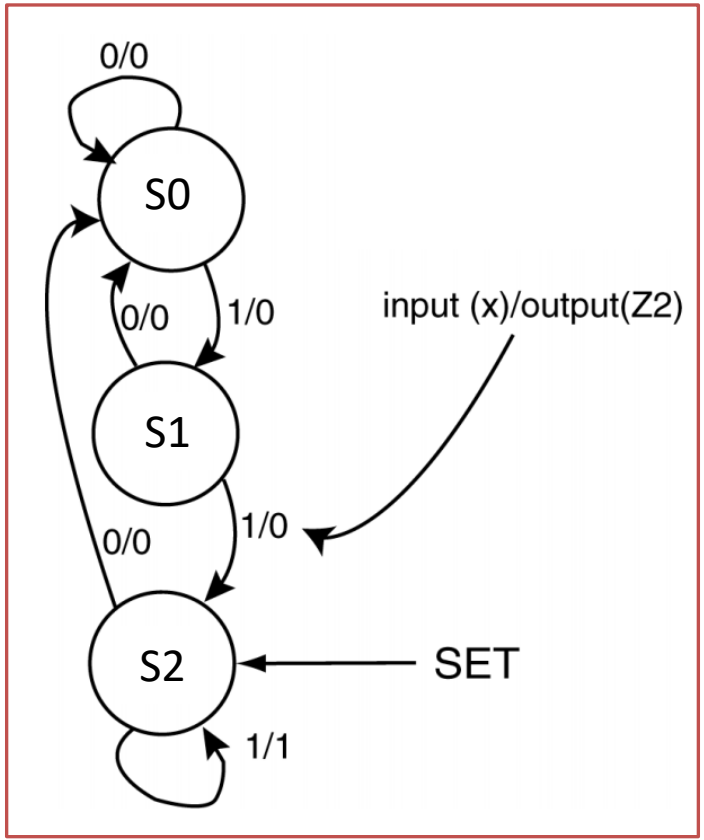
MAQUINA DE MEALY




```
architecture fsm3 of my_fsm3 is
   type state_type is (ST0,ST1,ST2);
   signal CS,NS : state_type;
begin

sync_proc: process(CLK,NS,SET)
begin
   if (SET = '1') then
      CS <= ST2;
   elsif (rising_edge(CLK)) then
      CS <= NS;
   end if;
end process sync_proc;
```

MAQUINA DE MEALY




```
comb_proc: process(CS,X)
begin
   case CS is
      Z2 <= '0'; -- pre-assign FSM outputs
      when ST0 => -- items regarding state ST0
         Z2 <= '0';
         if (X = '0') then
            NS <= ST0;
         else
            NS <= ST1;
         end if;
      when ST1 => -- items regarding state ST1
         Z2 <= '0';
         if (X = '0') then
            NS <= ST0;
         else
            NS <= ST2;
         end if;
      when ST2 => -- items regarding state ST2
         if (X = '0') then
            NS <= ST0;
            Z2 <= '0';
         else
            NS <= ST2;
            Z2 <= '1';
         end if;
      when others => -- the catch condition
            Z2 <= '1';
            NS <= ST0;
   end case;
end process comb_proc;
end fsm3;
```